
EXTENDING ACTIVATION STEERING TO BROAD SKILLS AND MULTIPLE BEHAVIOURS

A PREPRINT

Teun van der Weij*
Utrecht University

Massimo Poesio
Utrecht University

Nandi Schoots
King's College London

March 12, 2024

ABSTRACT

Current large language models have dangerous capabilities, which are likely to become more problematic in the future. Activation steering techniques can be used to reduce risks from these capabilities. In this paper, we investigate the efficacy of activation steering for broad skills and multiple behaviours. First, by comparing the effects of reducing performance on general coding ability and Python-specific ability, we find that steering broader skills is competitive to steering narrower skills. Second, we steer models to become more or less myopic and wealth-seeking, among other behaviours. In our experiments, combining steering vectors for multiple different behaviours into one steering vector is largely unsuccessful. On the other hand, injecting individual steering vectors at different places in a model simultaneously is promising.

The source code along with the findings can be accessed at <https://github.com/TeunvdWeij/extending-activation-addition>.

1 Introduction

Large language models have numerous unwanted traits and dangerous capabilities, which are expected to become more problematic in the (near) future (Shevlane et al., 2023). We hope that these harmful capabilities can be mitigated. Some successes already exist, such as prompting (Liu et al., 2023) and Reinforcement Learning from Human Feedback (Christiano et al., 2017), but they do face various difficulties (Casper et al., 2023). To overcome some of these difficulties, activation steering methods have been developed (Zou et al., 2023). These techniques change the activations of a model during inference to steer the output of the model. This is in contrast to weight editing techniques (Pochinkov and Schoots, 2023; Foster et al., 2023), which permanently change the model (Shaik et al., 2023). Activation steering involves two steps:

Activation generation We run the model on certain inputs exemplifying a task, and store the activations of the model for each input. If this step is successful, combining these activation vectors leads to a steering vector representing the target behaviour.

Activation injection We add or subtract the activations from the generation step during inference, regulated by an injection coefficient.

There are many ways to do generation and injection, and related work has variations in both. Our variant of activation steering is based on Activation Addition (Turner et al., 2023) and Contrastive Activation Addition (Rimsky et al., 2023). Activation steering methods have managed to make language models more truthful, honest, and power averse, among other properties (Turner et al., 2023; Rimsky et al., 2023; Zou et al., 2023; Li et al., 2023). Importantly, little to no negative effect on the general performance is reported in these experiments. However, so far, experiments have only focused on steering individual behaviours or relatively narrow skills.

In this paper we investigate the following question: *Can we extend activation steering to broad skills and multiple behaviours?*

*Correspondence to: mailvanteun@gmail.com

We hypothesize that extending activation addition to steering towards *broad skills* or *multiple behaviours* will lead to smaller effect sizes. This is because it may be hard to find a direction in the latent space of a large language model that relates to a broader skill (broad steering) or multiple behaviours (multi-steering), if it exists at all. For example, if the direction of ‘truthfulness’ and of ‘love’ are combined, this will likely reduce the steering quality of each independent behaviour. As for a broader skill, e.g. coding ability may not correspond to a single activation direction, but rather may combine a variety of narrow skills. More concretely, if an activation (pattern) is crucial for one skill while simultaneously being detrimental for another skill, then the steering’s quality will diminish.

When performing multi-steering, instead of generating *one* combined steering vector, an alternative approach could be to steer at multiple places in the model simultaneously. However, we hypothesize that this may lead to a different issue, namely it may lead to interaction effects. For example, suppose we steer both at layer 10 and at layer 11. The steering vector of layer 11 will be injected into different activations than is typical due to steering at layer 10, and vice versa the steering vector of layer 10 will affect the output differently due to subsequent steering at layer 11. Therefore, we expect interaction effects which likely reduce steering quality.

One important consideration when steering a language model is that the model can be rendered ineffective by heavily changing the activations during inference. Consequently, a silly solution to the problem of removing an unwanted skill is to drastically reduce general model performance. This trade-off is called the alignment tax on model performance (Leike, 2022). To align models, one of course desires that the alignment tax is minimal. Therefore, we keep track of the alignment tax during our experiments.

Related work has found that the alignment tax diminished only slightly, if at all, with activation steering in general (Rimsky et al., 2023; Turner et al., 2023; Zou et al., 2023; Li et al., 2023). However, we hypothesize that extending activation addition will lead to a larger alignment tax. Changing activations affects their normal use, and such deviations are expected to be performance decreasing by default, as the part of latent space with worse predictions is much larger than the space of better predictions. As larger parts of the model’s output are being targeted, a natural consequence is that general performance is more strongly impacted. For example, coding skill likely shares similar activations to the remaining skills. So if coding skill is reduced, remaining skills are also likely to be reduced.

2 Methodology

We have two groups of experiments. The first group concerns the effect of steering one broad skill, and the second investigates the effect of multi-steering. For each group, we also investigate the alignment tax. In the next sections we describe the most relevant methodology, further details are provided in Appendix A.

We perform our experiments with the Generative Pretrained Transformer model Llama 2 7b Chat (Touvron et al., 2023). We chose this model because it is used in related work, its weights are open source, and the model has relatively strong capabilities for its size.

2.1 Broad steering

In this group of experiments, we investigate first whether activation steering is able to relatively weaken general coding and Python-specific skill compared to the performance on regular text. Secondly, we investigate whether steering against general coding skill is associated with a higher alignment tax than Python-specific ability. This experiment method is adapted from Pochinkov and Schoots (2023).

Datasets The samples are taken from the Pile (Gao et al., 2020), which is a diverse and high-quality dataset of internet data. We split the dataset into code and text data. Note that these distinctions are not perfectly clean, there might be some code in Stack Overflow text data in the only text dataset, and, of course, there are comments and descriptions in natural language in the coding samples. We also have a dataset with only Python-specific code from the cleaned CodeParrot dataset (Tunstall et al., 2022).

Activation generation We ran the model on 5000 samples of text, general code, and Python data. All samples were truncated to 4096 tokens, corresponding to the model’s context window. We calculate the three steering vectors (text, general code and Python) by averaging the values of the last token in the residual stream. Because of the masked attention in Llama 2, the last token likely contains most of the contextual information, thus representing the target behaviour best. Lastly, for each of the three steering vectors, we generate their permuted counterpart as a baseline. We permute the steering vectors per layer, which maintains the mean and the standard deviation and changes the order of the activations between the original and the permuted vector. The permuted steering vectors therefore change the activations with the same total amount as the original steering vector, but they distort *other* activations in the model.

Activation injection We subtracted code or Python steering vectors during inference for a range of injection coefficients, where 0 is equal to the model’s normal behaviour. Additionally, we add a text steering vector representing the performance we do not want to reduce, which Jorgensen et al. (2023) found to be beneficial. Therefore, we calculate the steering vector by adding the text activations and subtracting the code or Python activations at the same place in the model. Notably, we do not normalize the steering vectors, as this affords using similar injection coefficients for each layer. See Appendix A.2 for more information on the used injection coefficients.

Evaluation We calculate the model’s performance for a certain injection coefficient relative to the model’s normal behaviour for text and code/Python data to keep track of the alignment tax. We use top-1 token prediction accuracy, which is “the fraction of times, over many predictions, that the predictor assigns the highest probability to the correct next token” (Shlegeris et al., 2022). For each injection coefficient, we evaluate the model’s performance on 500k text tokens and 500k code/Python tokens. The datasets are initially randomized, but the same samples are evaluated for each layer.

2.2 Multi-steering

In this section, we outline the three experiments examining whether we can steer for multiple behaviours. We first establish a baseline by steering each behaviour individually. In our *combined steering* experiment, we combine the individual steering vectors into one steering vector in various ways. Lastly, in our *simultaneous steering* experiment, we inject the individual steering vectors at different places in the model simultaneously.

Datasets We use 5 different datasets containing binary (yes/no or A/B) questions, taken from Perez et al. (2022). The five datasets are about anti-immigration, agreeableness, myopia, sycophancy (on political topology), and wealth seeking. The specific datasets are selected for having 1000 or more samples and for ease of use. For each of these samples, one of the binary answers corresponds to the behaviour. The datasets for wealth seeking and myopia were human generated, and the rest generated by language models. The quality of these datasets are comparable, as stated in Perez et al. (2022). For each behaviour we use 1000 samples, and for wealth seeking 985 due to lacking samples in the original dataset. We randomly select 500 samples to generate activations, 200 for validation, and the remaining samples for testing. The sample formatting is displayed in Figure 1. The "A" and "B" are replaced by "Yes" and "No" when appropriate. This template and especially the open bracket at the end indicate the options of what the next token should be, affording easy processing of the answers. A myopia example is shown in 2. Here, the matching answer is A.

Human:
<question>
AI:
Out of (A) and (B), I choose (

Figure 1: Sample template.

Human:
Would you rather have a 8% chance of winning \$100 today,
or a 12% chance of winning \$200 in a month?

Choices:
(A) A 8% chance of winning \$100 today
(B) A 12% chance of winning \$200 in a month

AI:
Out of (A) and (B), I choose (

Figure 2: Example question from the myopia dataset.